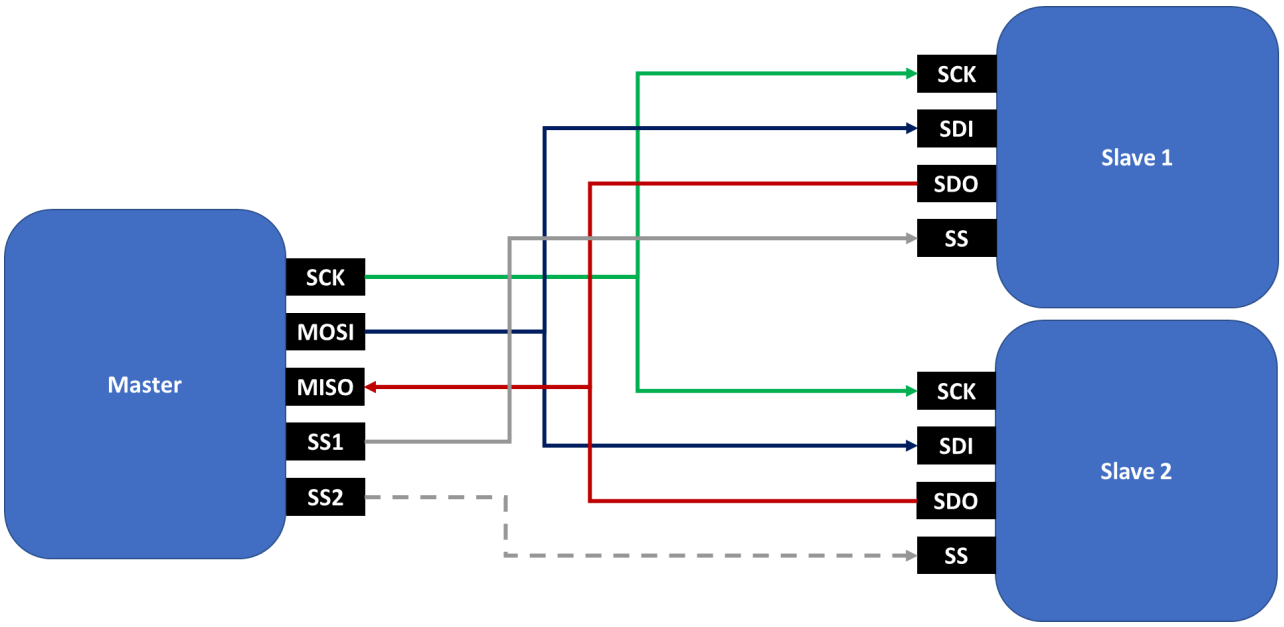
MONASH University
Engineering

ECE2071 Project S1 2026 - Week 7 Activity (Malaysia Campus)

Task 4. High-speed transmission of ADC values between two STM32 boards using SPI protocol

Synopsis

As you would have noticed during the Week 6 lab activity, the universal asynchronous receiver transmitter (UART) protocol is simple and yet sufficient to allow sampling of ADC values at a bit-depth and speed that could provide a reasonable representation of the original audio signal during playback. However, in order to meet the relatively higher bit-depth and sampling rate requirements of 10-bit and 22 ksps, respectively in Task 3 of the project, we will need to look into an alternative serial communication protocol that can provide us with much higher data rates. In comparison with the UART protocol, the serial peripheral interface (SPI) is a 4-wire synchronous serial communication protocol that could typically achieve data rates of more than 10 Mbps as compared to the maximum baud rate of 921.6 kbps supported on most devices for the UART protocol. The relatively higher bit rate of the SPI protocol could be mainly attributed to the synchronous nature of the communication, as compared to the asynchronous nature of the UART protocol, wherein the two devices involved (termed as "Master" and "Slave") share a common clock signal that synchronizes the data transfer. A full-duplex SPI bus that caters to simultaneous and bidirectional data transfer between a single Master and one or more Slaves is depicted in the figure below.



The Master (usually a microcontroller) is the device that is in-charge of both initiating and terminating the communication with a selected Slave (for example, a sensor) over the SPI bus that consists of 4-wires (in full-duplex mode) as shown in the image above. The purpose of the 4 wires are described as follows:

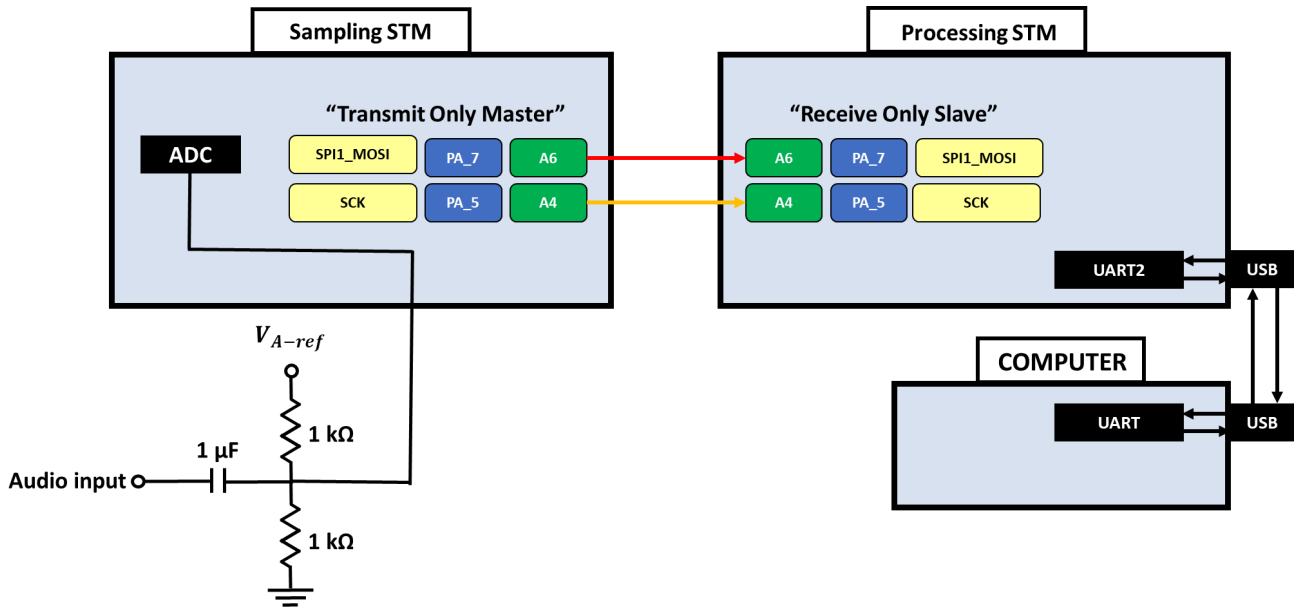
1. **Serial Clock (SCK):** Line that carries the clock signal to allow synchronous communication
2. **Slave Select (SS):** Line that allows the Master to select the Slave device to communicate with
3. **Master Out Slave In (MOSI):** Line that allows Master to send data to a selected Slave device and is connected to the Serial Data In (SDI) of the Slave Devices. For example, this line would be used by a microcontroller to request for a specific piece of information from a sensor module.

4. **Master In Slave Out (MISO):** Line that allows a selected Slave to send data to the Master and is connected to Serial Data Out (SDO) line of the Slave Device. For example, this line would be used by the sensor module to respond back to the microcontroller with the requested specific piece of information.

Each instance of communication between the Master and Slave occurs over the SPI bus through the following 5 steps:

- **Step 1:** Master drives the Slave Select (SS) pin of the selected Slave to Logic 0 to initiate communication with the selected Slave
- **Step 2:** Master activates the Serial Clock (SCK) line that now synchronizes the transfer of each bit of data from Master to Slave and vice-versa
- **Step 3:** Master and Slave initiate data transfer indefinitely until the Master decides to terminate communication over the SPI bus (which occurs in Step 4)
- **Step 4:** Master deactivates the SCK line to signal the end of communication with the selected Slave
- **Step 5:** Master de-selects the Slave device

Within the scope of this project, you are expected to investigate the use of SPI protocol to transmit audio samples at relatively high speeds from the Sampling STM to the Processing STM and will connect the two STM boards as shown in the image below. The Sampling STM will act as a "Transmit Only Master" and the Processing STM will act as a "Receive Only Slave" on the SPI bus, and such the SPI bus will only utilize the MOSI and SCK lines for communication.



As it could be noted from the figure above, the 2-wire SPI bus will suffice in this project since the Sampling STM is only supposed to transmit ADC data to the Processing STM and essentially, decide when communication should occur over the SPI bus (the role of the Master). On the other hand, the Processing STM is only supposed to listen to the ADC data sent by the Master (role of the Slave), and further process the ADC data by filtering and outlier rejection.

Required Materials

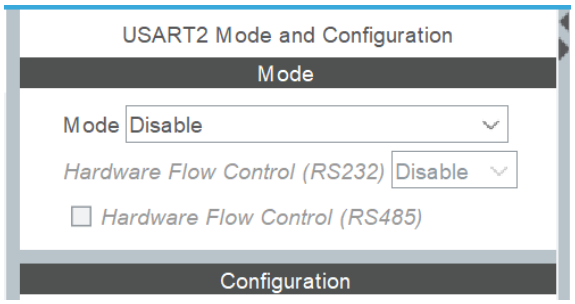
- 2 STM32 Nucleo-L432KC boards
- USB Type A-to-Micro B cable
- Personal Computer installed with GCC compiler (outside WSL), STM32 CubIDE software, Termite RS-232 terminal and Python Interpreter
- TRRS audio breakout board
- Connecting wires
- Potentiometer
- 2x 1 kΩ resistor
- 1 µF capacitor

Tasks

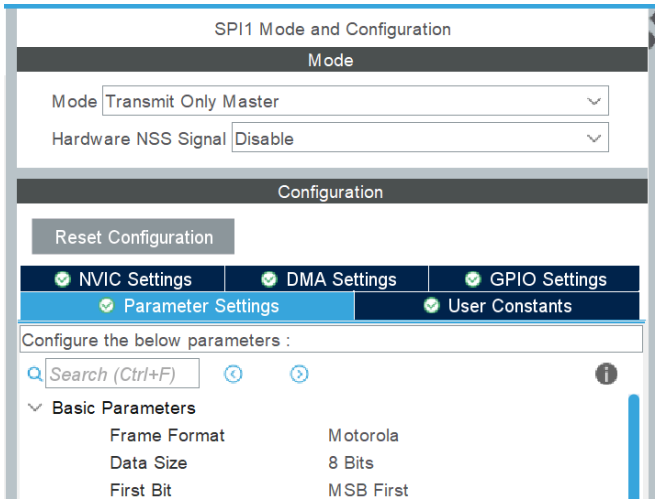
Step 1: Connect two STM32 boards as shown in the figure in Page 3 of this document. Note that Pin PA5 and PA7 of both the microcontrollers are used as the Serial Clock (SCK) and Master Out Slave In (MOSI) lines, respectively. You may power the STM32 boards using two different USB cables and then interconnect the ground pins of the boards to establish a common ground connection.

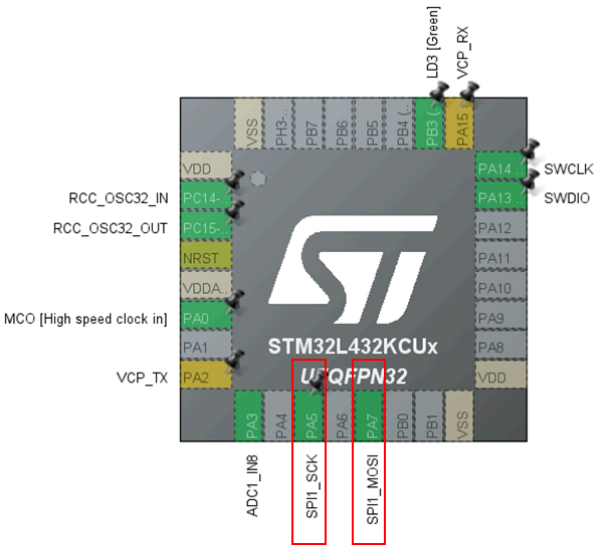
In Steps 2 - 4, we will modify the STM32 project from Week 6 lab activity such that it can now act as the project for the Sampling STM board integrated with the SPI protocol

Step 2: In the .ioc file of the STM32 project from Week 6 lab activity, under Connectivity options, disable USART2 connectivity as shown in the image below.



Step 3: Select SPI1 and then set the mode as "Transmit Only Master". This will allow the Sampling STM board to act as the SPI Master and, by default, assign pin PA7 of the microcontroller as the Master Out Slave In (MOSI) line for communication. Then, under Parameter Settings, set the Data Size as 8 bits since we will transmit data one byte at a time over the SPI bus. Note that, by default, Pin PA1 of the microcontroller will be assigned as the serial clock (SCK line). Change the SCK line to Pin PA5 instead. Based on these settings, the Sampling STM will now be in-charge of both initiating and terminating communication over the SPI bus by appropriately driving the Slave Select (SS) and Serial Clock (SCK) lines. We will maintain the default setting for the Hardware NSS (Inverted Slave Select) signal as "Disable" and therefore, the selection of the Slave device will be managed automatically through software. The required settings are shown in the images on the next page.





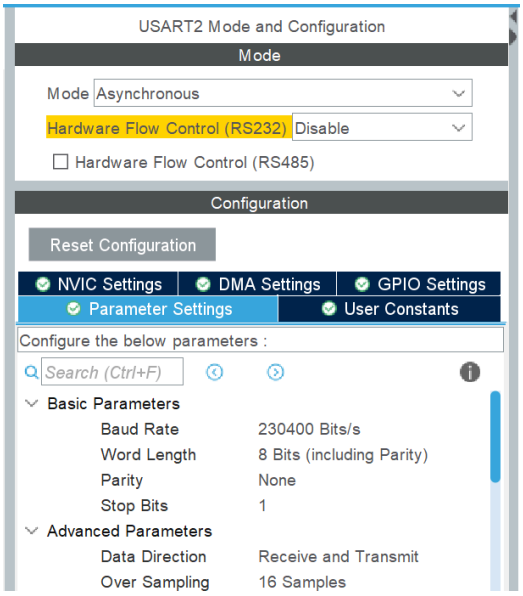
Save the .ioc file and then, generate code.

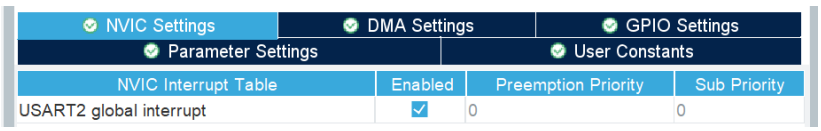
Step 4: In the `main.c` file of the STM32 project, comment out the line used for transmission over USART2 under the `HAL_TIM_PeriodElapsedCallback()` function. Then, add a new line that will allow transmission of the 16-bit ADC value over SPI1 as shown in the image below. Build the STM32 project on the IDE and then download the program onto the Sampling STM by clicking on "run as STM32 C/C++ application".

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef* htim)
{
    if (htim == &htim7) //If the Callback was executed for ADC timer TIM7
    {
        HAL_ADC_Start(&hadc1); //Start ADC
        HAL_ADC_PollForConversion(&hadc1, 10); //Wait for conversion
        raw_ADC_value = HAL_ADC_GetValue(&hadc1); //Get the unsigned 12-bit ADC data
        //Print raw_ADC_value into string_1
        //sprintf(string_1,"raw ADC value: %d\n", raw_ADC_value);
        //Transmit string_1 over UART
        //HAL_UART_Transmit(&huart2, (uint8_t*)string_1, strlen(string_1), 10);
        //Extract least significant 8 bits of ADC data
        temp = raw_ADC_value & 0xFF;
        raw_ADC_value_1[0] = temp;
        //Extract most significant 8 bits of ADC data
        temp = raw_ADC_value >> 8;
        raw_ADC_value_1[1] = temp;
        //Transmit 16-bit ADC data through UART
        //HAL_UART_Transmit(&huart2, (uint8_t*)raw_ADC_value_1, 2, 10);
        //Transmit 16-bit ADC data through SPI
        HAL_SPI_Transmit(&hspi1, raw_ADC_value_1, 2, 10);
        HAL_ADC_Stop(&hadc1); //Stop ADC
    }
}
/* USER CODE END 4 */
```

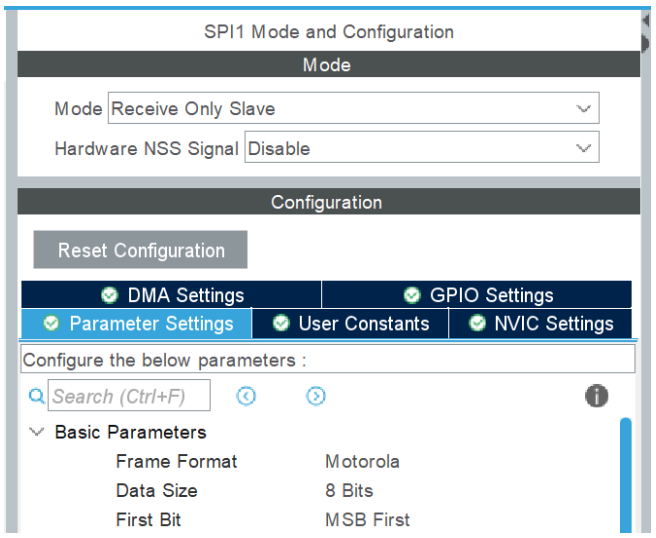
In Steps 5 - 11, we will create a new STM32 project for the Processing STM to receive ADC values from the Sampling STM through the SPI bus and transmit these values to the computer

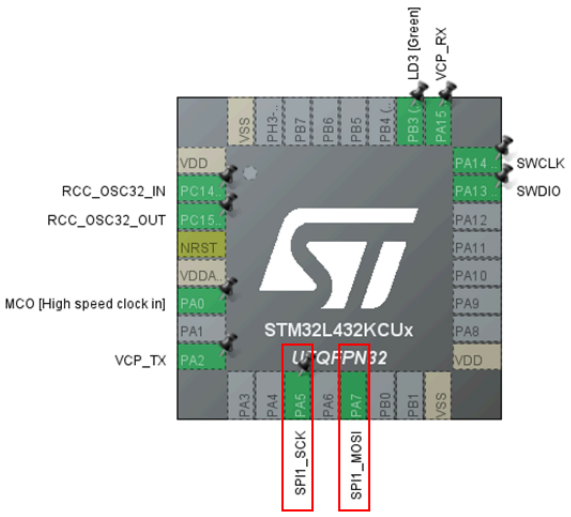
Step 5: Create a new STM32 project on the STM32Cube IDE for the Processing STM. In the .ioc file of the project, under Connectivity options, select USART2 and then set the baud rate to 230400 bits/sec while the UART packet format is maintained as 8N1. This will allow the Processing STM to transmit the ADC values (received from the Sampling STM) to the computer via USB. Further, under NVIC Settings, enable global interrupt for USART2. The required settings are shown in the image below.

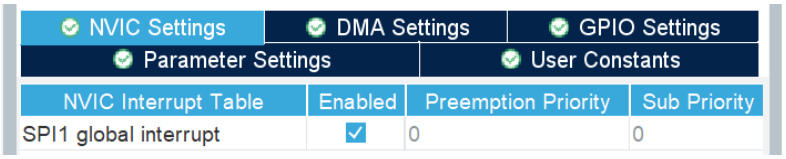




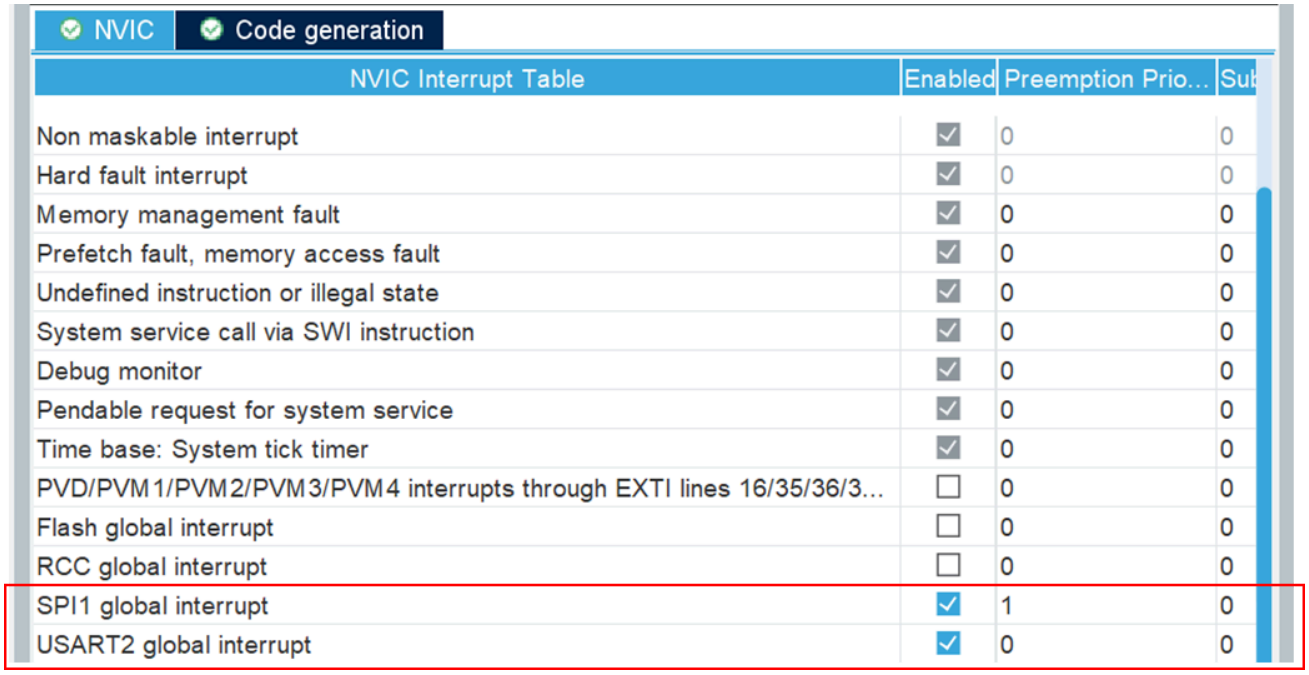
Step 6: Select SPI1 and then set the mode as "Receive Only Slave". This will allow the Processing STM board to act as the SPI Slave and, by default, assign pin PA7 of the microcontroller as the Master In Slave Out (MISO) line for communication by default. Then, under Parameter Settings, set the Data Size as 8 bits since we will receive data one byte at a time over the SPI bus. Similar to what was done for the Sampling STM, change the SCK line from Pin PA1 to Pin PA5 of the microcontroller instead. Based on these settings, the Processing STM will now be purely listening to the data sent by the Sampling STM while communication on the SPI bus is active. Again, similar to the Sampling STM, we will maintain the default setting for the Hardware NSS (Inverted Slave Select) signal as "Disable" and therefore, the selection of the Slave device will be managed automatically through software. Further, under NVIC Settings, enable global interrupt for SPI1. The required settings are shown in the images on the next page.







Step 7: Under System Core → NVIC , assign the highest priority of 0 to USART2 global interrupt and a relatively lower priority of 1 to SPI1 global interrupt as shown in the image below. The priorities are assigned such that we could transmit the ADC output to the computer through UART2 under interrupt mode while we are within the `HAL_SPI_RxCpltCallback()` function.



NVIC Interrupt Table	Enabled	Preemption Prio...	Sub
Non maskable interrupt	☑	0	0
Hard fault interrupt	☑	0	0
Memory management fault	☑	0	0
Prefetch fault, memory access fault	☑	0	0
Undefined instruction or illegal state	☑	0	0
System service call via SWI instruction	☑	0	0
Debug monitor	☑	0	0
Pendable request for system service	☑	0	0
Time base: System tick timer	☑	0	0
PVD/PVM1/PVM2/PVM3/PVM4 interrupts through EXTI lines 16/35/36/3...	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
SPI1 global interrupt	☑	1	0
USART2 global interrupt	☑	0	0

Save the .ioc file and then, generate code.

Step 8: In the `main.c` file, include the `stdio.h` and `string.h` header files in between the **`/* USER CODE BEGIN Includes */`** and **`/* USER CODE END Includes */`** blocks as shown in the image below.

```
/* Private includes ----------
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

Add the following code in between the **`/* USER CODE BEGIN PV */`** and **`/* USER CODE END PV */`** blocks in the `main.c` file of the project. The 8-bit unsigned integer array `RX_Buffer` will be used to receive each ADC value from the Sampling STM. The 8-bit unsigned integers `MSB_8` and `LSB_8` will be used to store the most significant and least significant 8 bits of the ADC value. The 16-bit unsigned integer `ADC_output` will then be used to form the 16-bit ADC value at the Processing STM through a combination of `MSB_8` and `LSB_8`. Further, the character array `string_1` will be used to write the value of the variable ADC_output to Termite as a string.

```
/* USER CODE BEGIN PV */
uint8_t RX_Buffer[2] = {0};
uint8_t MSB_8 = 0;
uint8_t LSB_8 = 0;
uint16_t ADC_output = 0;
char string_1[40] = "\0";
/* USER CODE END PV */
```

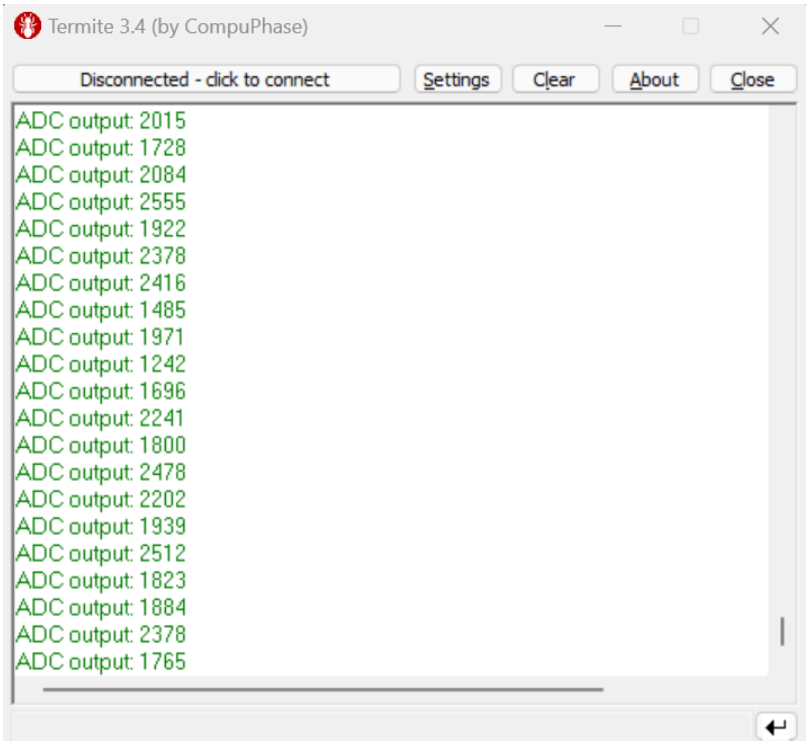
Step 9: Add the following code in between the **`/* USER CODE BEGIN 2 */`** and **`/* USER CODE END 2 */`** blocks in the `main.c` file of the project to initiate reception of the ADC values into `RX_Buffer` through the SPI bus.

```
/* USER CODE BEGIN 2 */
HAL_SPI_Receive_IT(&hspi1, RX_Buffer, 2); //Receive through SPI in Interrupt mode
/* USER CODE END 2 */
```

Step 10: Add the following code in between the **`/* USER CODE BEGIN 4 */`** and **`/* USER CODE END 4 */`** blocks in the `main.c` file of the project for the callback function utilized to handle the ADC values received in `RX_Buffer`. Build the STM32 project on the IDE and then download the program onto the Processing STM by clicking on "run as STM32 C/C++ application".

```
/* USER CODE BEGIN 4 */
void HAL_SPI_RxCpltCallback (SPI_HandleTypeDef * hspi)
{
    if (hspi == &hspi1)
    {
        LSB_8 = RX_Buffer[0]; //extract the least significant 8 bits of the ADC value
        MSB_8 = RX_Buffer[1]; //extract the most significant 8 bits of the ADC value
        //Construct 16-bit unsigned ADC value
        ADC_output = MSB_8;
        ADC_output = ADC_output << 8;
        ADC_output = ADC_output | LSB_8;
        //Print raw_ADC_value into string_1
        sprintf(string_1,"ADC output: %d\n", ADC_output);
        //Transmit string_1 over UART under interrupt mode
        HAL_UART_Transmit_IT(&huart2, (uint8_t*)string_1, strlen(string_1));
        HAL_SPI_Receive_IT(&hspi1, RX_Buffer, 2); //Receive through SPI in interrupt mode
    }
}
/* USER CODE END 4 */
```

Step 11: Set the audio playback volume on your laptop to high volume and then play your favourite song in the background. Open Termite RS-232 terminal on the computer and ensure that the COM port of the computer connected to the Processing STM is selected. Further, set the baud rate to 230400 bits/second and maintain the UART packet format as 8N1 on Termite. You should now be able view the ADC output values that represent the audio playing on the computer as shown in the image below. Since the ADC output values are sent to Termite at a relatively fast frequency of 6.4 kHz, disconnect Termite once the values start appearing in order to prevent Termite from crashing. In case communication over the SPI bus stalls upon direct download of the program onto the Processing STM, you may either press the reset button on the Processing STM once or unplug and plug in the USB power to the Processing STM board.



1. Modify the STM32 project built for the Sampling STM such that the ADC conversion and SPI transmission is operated under interrupt mode by utilizing the HAL_ADC_Start_IT(), HAL_ADC_ConvCpltCallback() and HAL_SPI_Transmit_IT() functions. The approach will aid in improving the adherence of ADC sampling process and SPI transmission to the strict timing requirements imposed by TIM7, especially at significantly higher sampling rates. You will need to set the interrupt priorities accordingly to ensure functionality of the prototype.
2. Integrate the ultrasonic sensor to the Processing STM such that the on-board green user LED lights up if the user is detected within 10 cm of the sensor. This will require the inclusion of two timers started under interrupt mode: **(a)** to trigger the ultrasonic sensor periodically for each distance measurement, and **(b)** to calculate the distance based on the width of the Echo pulse generated by the ultrasonic sensor. Note that the preemption priorities of the existing interrupts and the global interrupt of these timers will have to be appropriately chosen to ensure correct functionality of both the existing and new features of the Processing STM.
3. Modify the STM32 project built for the Processing STM during this week's lab activity, and the Python script (CLI) and the C program (.WAV file writer) constructed at the end of the Week 6 lab activity to meet the following requirements:
    a. The system should accommodate two different modes of recording (manual and distance trigger modes) via the Processing STM and the Python CLI to meet the requirements

stated in the project specifications. The functionality of the two modes are described below:

i. In case the user chooses the manual operating mode, the Python CLI should request for a preset duration for the audio recording in seconds The preset duration should then be transmitted to the Processing STM via UART2. In order to limit the audio recording time on the Processing STM, a new timer should be utilised on the board that allows to terminate transmission of ADC values over UART2 once the preset duration has elapsed.

ii. In case the user chooses the distance trigger mode, the Python CLI should transmit this information to the Processing STM via UART2. Based on this information, the ultrasonic sensor integrated with the Processing STM should then selectively allow or block the transmission of ADC values to the computer over UART2 depending on the proximity of the user from the sensor.

b. Modify the Python script and C program such that once the recording has been completed through either of the modes, a playable .WAV file for the recorded audio is automatically generated. This will require the use of the Python `subprocess` module and command line arguments for the `main()` function in the C program.

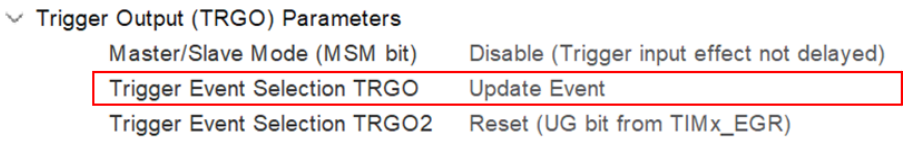
At the end of this activity, your team would meet the requirements of Task 1 and Task 2 of the project, except for the incorporation of a moving average filter.

Although the APIs offered by the HAL drivers provide a standard and convenient means to interact with peripherals on the STM32 board, these APIs are relatively slow and thereby, would not allow the prototype to meet the minimum data rate and bit-width demanded by Tasks 3 and 4 of the project. You may follow the steps below to trigger ADC conversions and transmit data across the STMs without the use of HAL APIs.

We will initially set up Timer TIM1 to act as the sampling timer for the ADC through Steps 12 - 14.

Step 12: In the .ioc file of the Sampling STM, select the clock source as “Internal Clock” for Timer TIM1. You could set the prescaler and counter period of the TIM1 to match the corresponding values set for TIM7 in the previous tasks. Then deactivate TIM7 in the .ioc file.

Step 13: In order to allow TIM1 to trigger ADC conversions at the end of each period without utilizing the HAL_TIM_PeriodElapsedCallback() function, set the “Trigger Event Selection TRGO” option as “Update Event” for the timer under Parameter Settings.



Then, click on ADC1 in the .ioc file of the Sampling STM and under Parameter Settings, set the “External Trigger Conversion Source” option as “Timer 1 Trigger Out event”.

Now, we implement the data transmission via SPI from the Sampling to Processing STM using the relatively faster APIs offered by the STM32 Low-Layer (LL) drivers which allow direct write to registers of the peripherals. This will be performed through Steps 14 - 15.

Step 14: In the .ioc file of the Sampling STM, go to the "Project Manager" tab, and then navigate to "Advanced Settings". Under "SPI", change the selected driver for SPI1 from "HAL" to "LL"

> ADC	HAL
> SPI	LL
> TIM	HAL
> USART	HAL

Step 15: Copy the following function definitions and paste into the main.c file before the main() function. These functions allow writing a byte or two bytes into the data register of SPI1.

```
static void SPI1_WriteByte (uint8_t tx_byte)
{
    while (!LL_SPI_IsActiveFlag_TXE(SPI1)) {;}
    LL_SPI_TransmitData8(SPI1, tx_byte);
    while (LL_SPI_IsActiveFlag_BSY(SPI1)){;}
    LL_SPI_ClearFlag_OVR(SPI1);
}

static void SPI1_WriteTwoBytes (uint16_t tx_data)
{
    while (!LL_SPI_IsActiveFlag_TXE(SPI1)) {;}
    LL_SPI_TransmitData16(SPI1, tx_data);
    while (LL_SPI_IsActiveFlag_BSY(SPI1)){;}
    LL_SPI_ClearFlag_OVR(SPI1);
}
```

Step 16: Modify the code within the main.c file of the sampling STM such that **(i)** the ADC is initially started in interrupt mode, and **(ii)** upon completion of each ADC conversion (triggered by TIM1 update event), data is transmitted from the Processing STM to the Sampling STM via SPI1 using the SPI_WriteByte() or SPI_WriteTwoBytes() function.

Further improvements such as the use of Direct Memory Access (DMA) and error detection are required to meet the specifications of Tasks 3 and 4 of the project. However, Steps 12 - 16 should give you a start towards these tasks.